In [ ]:
```
#Step 1: Import Libraries & Load Data
```

In [5]:
```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset
df = pd.read_csv("C:\\Users\\DEBABRATA\\Downloads\\archive (8)\\supply_chain_resilience_dataset.csv")
```

In [30]:
```
# Display the first 10 rows to inspect the structure
print("Dataset Shape:", df.shape)
df.head(10)
```

```
Dataset Shape: (1000, 26)
```

Out[30]:

	Order_ID	Buyer_ID	Supplier_ID	Product_Category	Quantity_Ordered	Order_Date	Dispatch_Date	Delivery_Date	Shipping_M
0	O1000	B33	S23	Textiles	469	2023-10-24	2023-10-27	2023-10-28	
1	O1001	B1	S20	Machinery	365	2023-07-07	2023-07-08	2023-07-09	F
2	O1002	B2	S10	Food	333	2023-12-28	2023-12-29	2024-01-07	
3	O1003	B6	S10	Machinery	142	2023-01-14	2023-01-17	2023-01-20	
4	O1004	B5	S4	Machinery	897	2023-01-12	2023-01-14	2023-01-16	F
5	O1005	B9	S11	Pharma	814	2023-03-28	2023-03-30	2023-04-08	
6	O1006	B34	S30	Food	849	2023-01-12	2023-01-14	2023-01-23	F
7	O1007	B7	S30	Food	136	2023-01-25	2023-01-27	2023-02-05	
8	O1008	B16	S24	Food	115	2023-10-28	2023-10-29	2023-11-07	
9	O1009	B26	S13	Machinery	613	2023-08-31	2023-09-01	2023-09-10	

10 rows × 26 columns

In [ ]:
```
#tep 2: Data Cleaning
```

In [8]:
```
# 1. Check for missing values before cleaning
print("--- Missing Values Before Cleaning ---")
print(df.isnull().sum()[df.isnull().sum() > 0])
```

```
--- Missing Values Before Cleaning ---
Disruption_Type        486
Disruption_Severity    486
dtype: int64
```

In [10]:
```
# 2. Impute missing values for disruption columns (None for non-disrupted orders)
df['Disruption_Type'] = df['Disruption_Type'].fillna('None')
df['Disruption_Severity'] = df['Disruption_Severity'].fillna('None')
```

In [11]:
```
# 3. Convert date columns to datetime objects
date_columns = ['Order_Date', 'Dispatch_Date', 'Delivery_Date']
for col in date_columns:
    df[col] = pd.to_datetime(df[col])
```

In [12]:
```
# 4. Verify cleaning results
print("\n--- Missing Values After Cleaning ---")
print("Total missing values:", df.isnull().sum().sum())
print("\nData Types:")
print(df[date_columns].dtypes)
```

```
--- Missing Values After Cleaning ---
Total missing values: 0

Data Types:
Order_Date       datetime64[ns]
Dispatch_Date    datetime64[ns]
Delivery_Date    datetime64[ns]
dtype: object
```

In [ ]:
```
#Step 3:Exploratory Data Analysis & Summary
```

In [14]:
```
# 1. Statistical Summary of Key Numeric Features
key_columns = ['Quantity_Ordered', 'Order_Value_USD', 'Delay_Days', 'Supplier_Reliability_Score', 'Supply_Risk_Flag']
summary_stats = df[key_columns].describe()
```

```
print("--- Summary Statistics ---")
display(summary_stats)
```

```
--- Summary Statistics ---
```

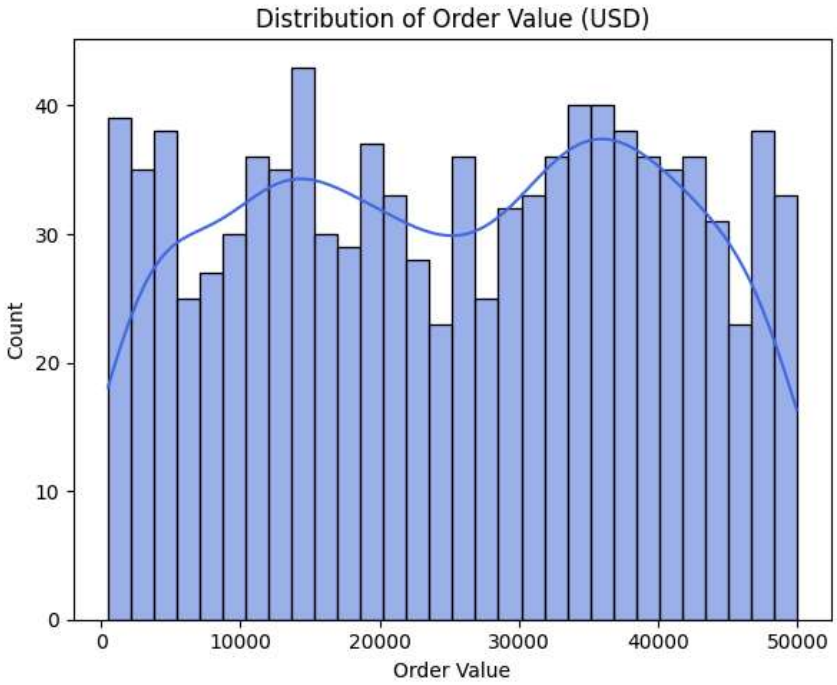
	Quantity_Ordered	Order_Value_USD	Delay_Days	Supplier_Reliability_Score	Supply_Risk_Flag
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	504.288000	25290.073670	1.943000	0.745760	0.514000
std	295.545455	14380.329641	2.852699	0.143063	0.500054
min	10.000000	500.580000	0.000000	0.500000	0.000000
25%	248.000000	12891.837500	0.000000	0.620000	0.000000
50%	499.000000	25596.005000	1.000000	0.740000	1.000000
75%	760.500000	37574.430000	3.000000	0.870000	1.000000
max	999.000000	49945.180000	10.000000	1.000000	1.000000

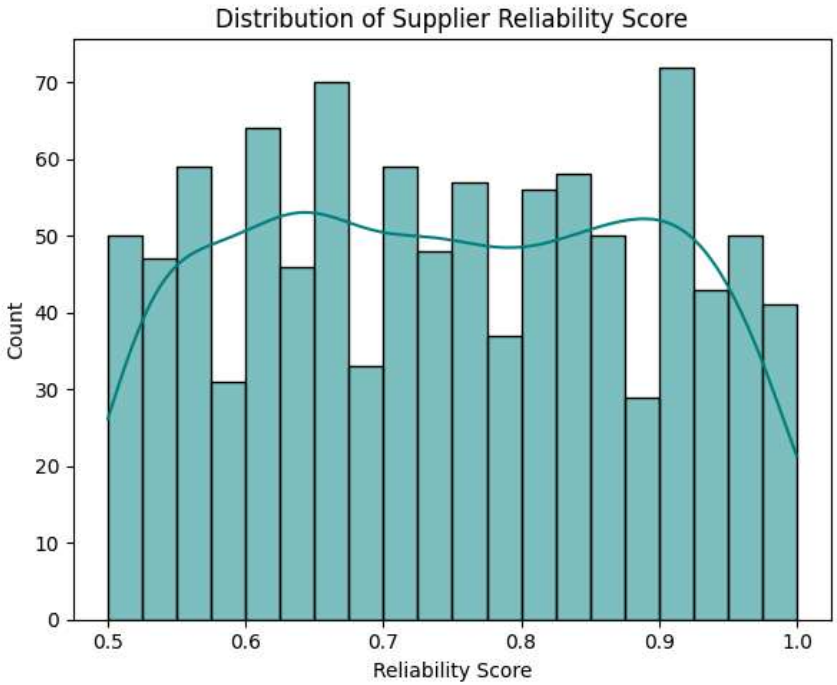
In [15]:
```
# 2. Visualizing Distributions (Histograms)
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
sns.histplot(df['Order_Value_USD'], kde=True, color='royalblue', bins=30)
plt.title('Distribution of Order Value (USD)')
plt.xlabel('Order Value')

plt.subplot(1, 2, 2)
sns.histplot(df['Supplier_Reliability_Score'], kde=True, color='teal', bins=20)
plt.title('Distribution of Supplier Reliability Score')
plt.xlabel('Reliability Score')

plt.tight_layout()
plt.show()
```





In [16]:
```
#Step 4:Statistical Analysis
```

In [18]:
```
# 1. Correlation Matrix for Numeric Features
numeric_cols = df.select_dtypes(include=[np.number]).columns
correlation_matrix = df[numeric_cols].corr()

# Plotting Correlation Heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix[['Supply_Risk_Flag']].sort_values(by='Supply_Risk_Flag', ascending=False),
            annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Correlation with Supply Risk Flag')
plt.show()
```



In [19]:
```
# 2. Groupby Analysis: Average Delay & Risk Rate by Product Category
category_analysis = df.groupby('Product_Category').agg(
    Total_Orders=('Order_ID', 'count'),
    Average_Delay_Days=('Delay_Days', 'mean'),
    Average_Order_Value=('Order_Value_USD', 'mean'),
    Risk_Rate=('Supply_Risk_Flag', 'mean')
```

```
).reset_index()

print("--- Aggregated Metrics by Product Category ---")
display(category_analysis)
```

```
--- Aggregated Metrics by Product Category ---
```

	Product_Category	Total_Orders	Average_Delay_Days	Average_Order_Value	Risk_Rate
0	Electronics	210	2.076190	27090.421429	0.509524
1	Food	190	1.705263	24785.769105	0.531579
2	Machinery	204	1.735294	24676.614706	0.490196
3	Pharma	190	2.078947	24334.138895	0.500000
4	Textiles	206	2.106796	25409.093447	0.538835

In [20]:
```
#Step 5: Advanced Grouping & Pivot Tables
```

In [21]:
```
# 1. Multi-level Groupby (Shipping Mode & Product Category)
multi_group = df.groupby(['Shipping_Mode', 'Product_Category']).agg(
    Mean_Delay=('Delay_Days', 'mean'),
    Total_Value=('Order_Value_USD', 'sum'),
    Risk_Percentage=('Supply_Risk_Flag', lambda x: (x.sum() / x.count()) * 100)
).reset_index()

print("--- Multi-Level Groupby: Shipping Mode vs Product Category ---")
display(multi_group)
```

```
--- Multi-Level Groupby: Shipping Mode vs Product Category ---
```

	Shipping_Mode	Product_Category	Mean_Delay	Total_Value	Risk_Percentage
0	Air	Electronics	2.085106	1459631.23	51.063830
1	Air	Food	1.264151	1240212.81	47.169811
2	Air	Machinery	0.980769	1391999.78	38.461538
3	Air	Pharma	1.842105	1491095.22	43.859649
4	Air	Textiles	2.648148	1361338.69	57.407407
5	Rail	Electronics	1.742424	1715043.06	50.000000
6	Rail	Food	1.977273	1014011.13	59.090909
7	Rail	Machinery	1.901639	1501506.79	50.819672
8	Rail	Pharma	2.692308	894201.58	53.846154
9	Rail	Textiles	2.133333	1035167.82	57.777778
10	Road	Electronics	2.220000	1375979.17	52.000000
11	Road	Food	1.957447	1324881.68	53.191489
12	Road	Machinery	2.086207	1316082.63	51.724138
13	Road	Pharma	1.977273	1021988.47	52.272727
14	Road	Textiles	1.561404	1448775.17	47.368421
15	Sea	Electronics	2.382979	1138335.04	51.063830
16	Sea	Food	1.695652	1130190.51	54.347826
17	Sea	Machinery	2.000000	824440.20	57.575758
18	Sea	Pharma	1.960000	1216201.12	52.000000
19	Sea	Textiles	2.120000	1388991.57	54.000000

In [22]:
```
# 2. Pivot Table: Average Delay Days by Shipping Mode and Disruption Type
pivot_table = pd.pivot_table(
    df,
```

```
    values='Delay_Days',
    index='Shipping_Mode',
    columns='Disruption_Type',
    aggfunc='mean',
    fill_value=0
)

print("\n--- Pivot Table: Mean Delay Days ---")
display(pivot_table)
```

```
--- Pivot Table: Mean Delay Days ---
```

Disruption_Type	Customs	None	Shortage	Strike	Weather
Shipping_Mode					
Air	3.406250	0.0	3.600000	3.843750	4.000000
Rail	3.500000	0.0	3.684211	4.088235	3.837838
Road	3.848485	0.0	4.500000	3.000000	4.000000
Sea	4.225806	0.0	3.400000	4.047619	3.676471

In [23]:
```
#Step 6: Temporal (Date/Time) Analysis
```

In [24]:
```
# 1. Calculate Order Fulfillment Duration (Dispatch to Delivery)
df['Fulfillment_Duration'] = (df['Delivery_Date'] - df['Dispatch_Date']).dt.days

# 2. Extract Month from Order Date for Trend Analysis
df['Order_Month'] = df['Order_Date'].dt.to_period('M')

# 3. Monthly Trend Aggregation
monthly_trend = df.groupby('Order_Month').agg(
    Total_Orders=('Order_ID', 'count'),
    Average_Delay=('Delay_Days', 'mean'),
    Total_Order_Value=('Order_Value_USD', 'sum')
).reset_index()

monthly_trend['Order_Month'] = monthly_trend['Order_Month'].astype(str)
print("--- Monthly Supply Chain Trend ---")
display(monthly_trend)
```

```
--- Monthly Supply Chain Trend ---
```

	Order_Month	Total_Orders	Average_Delay	Total_Order_Value
0	2023-01	89	1.730337	2132445.53
1	2023-02	61	1.557377	1508064.84
2	2023-03	86	2.011628	2531121.74
3	2023-04	88	2.159091	2249775.71
4	2023-05	100	1.700000	2559263.40
5	2023-06	83	1.722892	2141365.22
6	2023-07	67	1.746269	1699460.58
7	2023-08	94	2.191489	2252150.17
8	2023-09	82	2.463415	2126346.94
9	2023-10	92	1.804348	2447935.48
10	2023-11	76	2.157895	1683927.60
11	2023-12	82	1.987805	1958216.46

In [25]:
```
#Step 7: Prediction Model Syntax (Random Forest Classifier)
```

In [28]:
```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# 1. Define Features (X) and Target (y)
features = [
    'Quantity_Ordered', 'Order_Value_USD', 'Delay_Days',
    'Historical_Disruption_Count', 'Supplier_Reliability_Score',
    'Dominant_Buyer_Flag', 'Parameter_Change_Magnitude'
]

X = df[features]
y = df['Supply_Risk_Flag']
```

```
# 2. Split Dataset into Training (80%) and Testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Train Random Forest Classifier
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# 4. Predict and Evaluate
y_pred = rf_model.predict(X_test)

print(f"Model Accuracy: {accuracy_score(y_test, y_pred):.4f}\n")
print("Classification Report:")
print(classification_report(y_test, y_pred))

# 5. Feature Importance Visualization
importances = pd.Series(rf_model.feature_importances_, index=features).sort_values(ascending=True)

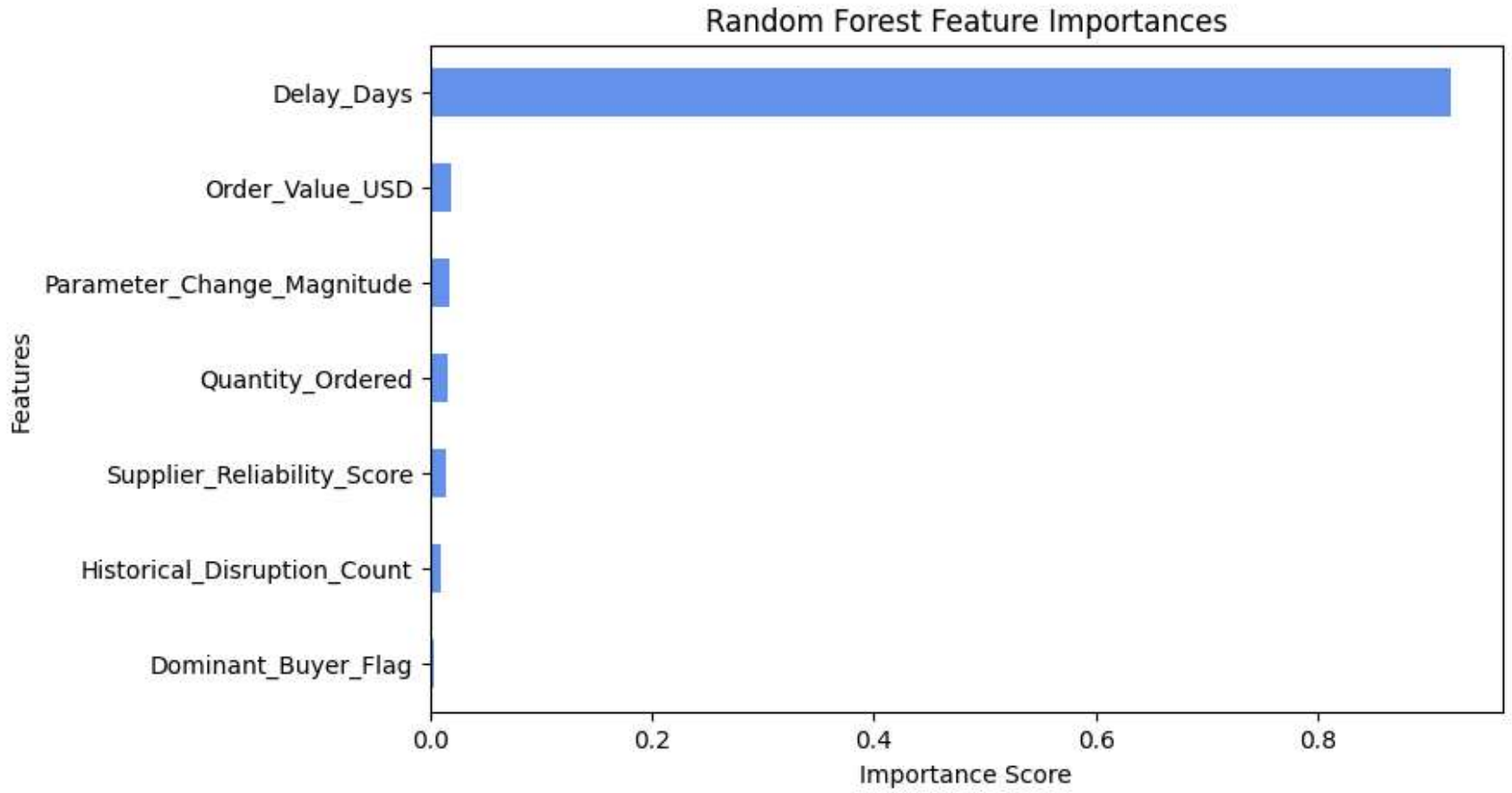
plt.figure(figsize=(8, 5))
importances.plot(kind='barh', color='cornflowerblue')
plt.title('Random Forest Feature Importances')
plt.xlabel('Importance Score')
plt.ylabel('Features')
plt.show()
```

```
Model Accuracy: 1.0000

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        92
           1       1.00      1.00      1.00       108

    accuracy                           1.00       200
   macro avg       1.00      1.00      1.00       200
weighted avg       1.00      1.00      1.00       200
```


Random Forest Feature Importances
Delay_Days
Order_Value_USD
Parameter_Change_Magnitude
Quantity_Ordered
Supplier_Reliability_Score
Historical_Disruption_Count
Dominant_Buyer_Flag
Features
0.0
0.2
0.4
0.6
0.8
Importance Score

In [ ]: